SQL Injection (SQLi)

Week 6 Notes | OWASP Top 10 | DG Cyber Security Internship | Diksha Uma Shankar Rao

What it is

SQL Injection is an injection flaw where untrusted user input is placed directly into a SQL query. The database cannot tell the difference between the query written by the developer and the data typed by the user, so an attacker can add SQL syntax that changes what the query does. It belongs to the Injection category of the OWASP Top 10.

How it works (example from the DVWA lab)

The vulnerable page builds its query by joining the user's input into a string:

```
SELECT first_name, last_name FROM users
WHERE user_id = '$id';
```

If the user enters `1' OR '1'='1`, the query becomes `WHERE user_id = '1' OR '1'='1'`. The condition is always true, so every row in the users table is returned instead of one user.

Main types

- **In-band:** results come back on the same page. Includes error-based and UNION-based injection.
- **Blind:** no data is shown, so the attacker infers answers from true/false page behaviour (boolean-based) or response delays (time-based).
- **Out-of-band:** data is sent through a different channel, such as a DNS or HTTP request from the database server.

Impact

- Reading sensitive data such as usernames, password hashes and personal details
- Bypassing login pages
- Modifying or deleting data
- In severe cases, running commands on the database server

How it was tested in this project

- Tool: DVWA (Security level Low) with Burp Suite capturing the traffic
- Payload: `1' OR '1'='1` in the User ID field of the SQL Injection module
- Result: all user records were displayed. Risk rating: High

Prevention

- **Parameterised queries / prepared statements** so input is always treated as data. This is the primary fix.
- Use an ORM or stored procedures that do not build SQL from strings
- Validate input on the server with an allow-list (type, length, format)
- Give the application database account the least privilege it needs
- Show generic error messages, never raw database errors
- A web application firewall can add a layer of defence but is not a replacement for safe code

Key takeaway

Never build SQL by concatenating user input. Separate code from data using parameterised queries.